Rutas de la API InWork

Backend principal en FastAPI + PostgreSQL · 135 rutas · cada ejemplo de esta guía se probó contra la API con los datos de prueba de DB.sql

1. Cómo se usa

Dirección de la API. En tu PC: `http://localhost:8000` (se enciende con `uvicorn main:app --reload`). En la nube: la URL de tu servicio. En `/docs` está Swagger para probar todo desde el navegador.

Paso 1 — iniciar sesión. `POST /auth/login` con el correo y la contraseña. La respuesta trae `access_token`, el nombre y el rol.

Paso 2 — mandar el token. Todas las rutas (menos registro, login, token y `/`) piden este encabezado:

```
Authorization: Bearer <access_token>
```

En Svelte (JavaScript) queda así:

```
const API = 'http://localhost:8000';

// Login
const r = await fetch(`${API}/auth/login`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ correo, password })
});
const sesion = await r.json();          // sesion.access_token, sesion.rol, sesion.nombre

// Cualquier otra ruta
const trabajos = await fetch(`${API}/trabajos/`, {
  headers: { Authorization: `Bearer ${sesion.access_token}` }
}).then(r => r.json());

// Crear (POST) o editar (PUT) con body
await fetch(`${API}/trabajos/`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json', Authorization: `Bearer ${sesion.access_token}` },
  body: JSON.stringify(datosDelFormulario)
});
```

Lo que responde la API

Tipo de ruta	Respuesta
GET lista	Un arreglo `[ {...}, {...} ]`
GET por id	Un objeto `{...}`
POST (crear)	`{"mensaje": "...", "id": 27}` — el id del registro nuevo
PUT (editar) y DELETE	`{"mensaje": "..."}`
Cualquier error	`{"detail": "texto del error"}` — muestra ese texto al usuario

Código	Qué significa
200	Todo bien.
400	Dato no permitido o regla de la base de datos (ej: fecha pasada, oferta en tu propio trabajo, correo repetido).
401	Sin token, token vencido o el rol cambió: hay que iniciar sesión de nuevo.
403	Tu rol no tiene permiso o el registro no es tuyo.
404	No existe (o está desactivado).

Código	Qué significa
422	El body está mal: falta un campo obligatorio o un tipo no coincide. `detail` dice cuál.
503	La API de pagos (Express) no responde.

Formatos y reglas generales

- Fechas `AAAA-MM-DD` (ej: `2026-11-20`), horas `HH:MM`, fecha y hora `AAAA-MM-DDTHH:MM:SS`.
- Los campos opcionales se pueden omitir o mandar como `null`.
- DELETE no borra de verdad: pone `estado = false` (borrado lógico) y el registro deja de salir en las listas.
- Todas las tablas traen `created`, `updated` y `estado`.
- Las rutas terminan en `/` cuando son la lista o el crear: `/trabajos/`.
- **IDs en el frontend:** la API devuelve ids y el frontend los usa por dentro para pedir los nombres. Carga los catálogos (categorías, ciudades, habilidades) una vez y busca el nombre por id; para el nombre de otra persona usa `GET /usuarios/{id}/perfil`. En pantalla solo se muestran nombres.

2. Roles y usuarios de prueba

Cada cuenta tiene un solo rol (`usuarios.id_rol`). Empleado y Empleador se pueden cambiar entre sí con `POST /auth/cambiar-rol`. La contraseña de todos los usuarios de prueba es `Inwork2026*`.

id	Rol	Qué hace	Usuario de prueba
1	**Administrador**	Acceso total. Crea usuarios, cambia roles y contraseñas, maneja roles, módulos y permisos, ve la auditoría.	`sebastian.rojas@inwork.com.co`
2	**Moderador**	Revisa reportes y verificaciones de identidad, maneja catálogos y notificaciones, ve casi todo.	`mariana.castro@inwork.com.co`
3	**Analista**	Solo lectura: ve usuarios, trabajos, contratos, reportes, pagos e informes para sacar estadísticas.	`jorge.pertuz@inwork.com.co`
4	**Empleador**	Publica trabajos, recibe ofertas, acepta una y contrata, paga y califica.	`valentina.mendoza93@gmail.com`
5	**Empleado**	Busca trabajos, envía ofertas, hace el trabajo, recibe el pago y califica.	`kevin.escorcia.elec@gmail.com`

Permisos de cada rol por módulo (tabla rol_modulo)

C = crear, R = leer, U = editar, D = eliminar. Es lo que devuelve `GET /auth/mis-modulos` para armar el menú y mostrar u ocultar botones. El Administrador puede todo aunque no esté en la tabla. Además de este permiso, muchas rutas solo dejan tocar los registros propios (se indica en cada ruta).

Módulo	Administrador	Moderador	Analista	Empleador	Empleado
`roles`	CRUD	—	—	—	—
`ubicacion`	CRUD	CRUD	R	R	R
`categorias`	CRUD	CRUD	R	R	R
`usuarios`	CRUD	R	R	RUD	CRUD
`verificaciones`	CRUD	RUD	R	CRU	CRU
`trabajos`	CRUD	R	R	CRUD	R
`ofertas`	CRUD	R	R	R	CRUD
`contrataciones`	CRUD	R	R	CRU	RU
`calificaciones`	CRUD	R	R	CRUD	CRUD
`chat`	CRUD	R	—	CRUD	CRUD
`notificaciones`	CRUD	CRUD	R	RUD	RUD
`adjuntos`	CRUD	RD	—	CRUD	CRUD
`reportes`	CRUD	RUD	R	CRU	CRU
`seguridad`	CRUD	—	—	—	—
`auditoria`	CRUD	—	—	—	—
`pagos`	CRUD	RU	R	CRU	CRU
`informes`	CRUD	—	R	—	—

Flujo principal de InWork

#	Quién	Qué hace	Ruta
1	**Empleador**	publica un trabajo	`POST /trabajos/`
2	**Empleado**	busca trabajos (todos o cercanos) y envía una oferta	`GET /trabajos/cercanos`, `POST /ofertas/`
3	**Empleador**	ve las ofertas de su trabajo y acepta una: se crea la contratación y la custodia del pago	`GET /ofertas/`, `POST /ofertas/{id}/aceptar`
4	**Empleador**	paga en la API de pagos (Express); el dinero queda asegurado	`API de pagos: /api/custodias/{id}/depositar`
5	**Los dos**	conversan por chat mientras se hace el trabajo	`POST /chats/`, `POST /mensajes/`
6	**Los dos**	confirman en la API de pagos; InWork le paga al trabajador	`API de pagos: /api/custodias/{id}/confirmar`
7	**Empleador**	finaliza la contratación	`POST /contrataciones/{id}/finalizar`
8	**Los dos**	se califican de 1 a 5	`POST /calificaciones/`

3. Índice de rutas

Todas las rutas en una sola lista. El detalle de cada una (body, ejemplo y respuesta) está en la sección 4.

Método	Ruta	Quién la usa
Estado de la API		
GET	/	Cualquiera, sin token
Autenticación		
POST	/auth/registro	Cualquiera, sin token
POST	/auth/login	Cualquiera, sin token
POST	/auth/token	Cualquiera, sin token
POST	/auth/cambiar-rol	Empleador, Empleado
GET	/auth/me	Todos los roles (con token)
GET	/auth/mis-modulos	Todos los roles (con token)
PUT	/auth/cambiar-password	Todos los roles (con token)
Roles		
GET	/roles/	Administrador
GET	/roles/{id_rol}	Administrador
POST	/roles/	Administrador
PUT	/roles/{id_rol}	Administrador
DELETE	/roles/{id_rol}	Administrador
Módulos		
GET	/modulos/	Administrador
GET	/modulos/{id_modulo}	Administrador
POST	/modulos/	Administrador
PUT	/modulos/{id_modulo}	Administrador
DELETE	/modulos/{id_modulo}	Administrador
Permisos por rol		
GET	/rol-modulo/	Administrador
GET	/rol-modulo/{id_rol_modulo}	Administrador
POST	/rol-modulo/	Administrador
PUT	/rol-modulo/{id_rol_modulo}	Administrador
DELETE	/rol-modulo/{id_rol_modulo}	Administrador
Países		
GET	/paises/	Administrador, Moderador, Analista, Empleador, Empleado
GET	/paises/{id_pais}	Administrador, Moderador, Analista, Empleador, Empleado
POST	/paises/	Administrador, Moderador
PUT	/paises/{id_pais}	Administrador, Moderador
DELETE	/paises/{id_pais}	Administrador, Moderador

Método	Ruta	Quién la usa
Ciudades		
GET	/ciudades/	Administrador, Moderador, Analista, Empleador, Empleado
GET	/ciudades/{id_ciudad}	Administrador, Moderador, Analista, Empleador, Empleado
POST	/ciudades/	Administrador, Moderador
PUT	/ciudades/{id_ciudad}	Administrador, Moderador
DELETE	/ciudades/{id_ciudad}	Administrador, Moderador
Categorías		
GET	/categorias/	Administrador, Moderador, Analista, Empleador, Empleado
GET	/categorias/{id_categoria}	Administrador, Moderador, Analista, Empleador, Empleado
POST	/categorias/	Administrador, Moderador
PUT	/categorias/{id_categoria}	Administrador, Moderador
DELETE	/categorias/{id_categoria}	Administrador, Moderador
Habilidades		
GET	/habilidades/	Administrador, Moderador, Analista, Empleador, Empleado
GET	/habilidades/{id_habilidad}	Administrador, Moderador, Analista, Empleador, Empleado
POST	/habilidades/	Administrador, Moderador
PUT	/habilidades/{id_habilidad}	Administrador, Moderador
DELETE	/habilidades/{id_habilidad}	Administrador, Moderador
Usuarios		
GET	/usuarios/	Administrador, Moderador, Analista
GET	/usuarios/{id_usuario}	Administrador, Moderador, Analista, Empleador, Empleado
GET	/usuarios/{id_usuario}/perfil	Todos los roles (con token)
POST	/usuarios/	Administrador
PUT	/usuarios/{id_usuario}	Administrador, Empleador, Empleado
PUT	/usuarios/{id_usuario}/rol	Administrador
PUT	/usuarios/{id_usuario}/password	Administrador
DELETE	/usuarios/{id_usuario}	Administrador, Empleador, Empleado
Habilidades de usuarios		
GET	/usuarios-habilidades/	Administrador, Moderador, Analista, Empleador, Empleado
GET	/usuarios-habilidades/{id_usuario_habilidad}	Administrador, Moderador, Analista, Empleador, Empleado
POST	/usuarios-habilidades/	Administrador, Empleado
PUT	/usuarios-habilidades/{id_usuario_habilidad}	Administrador, Empleador, Empleado
DELETE	/usuarios-habilidades/{id_usuario_habilidad}	Administrador, Empleador, Empleado

Método	Ruta	Quién la usa
Verificación de identidad		
GET	/verificaciones-identidad/	Administrador, Moderador, Analista, Empleador, Empleado
GET	/verificaciones-identidad/{id_verificacion}	Administrador, Moderador, Analista, Empleador, Empleado
POST	/verificaciones-identidad/	Administrador, Empleador, Empleado
PUT	/verificaciones-identidad/{id_verificacion}	Administrador, Moderador, Empleador, Empleado
PUT	/verificaciones-identidad/{id_verificacion}/revisar	Administrador, Moderador
DELETE	/verificaciones-identidad/{id_verificacion}	Administrador, Moderador
Trabajos		
GET	/trabajos/	Administrador, Moderador, Analista, Empleador, Empleado
GET	/trabajos/cercanos	Administrador, Moderador, Analista, Empleador, Empleado
GET	/trabajos/{id_trabajo}	Administrador, Moderador, Analista, Empleador, Empleado
POST	/trabajos/	Administrador, Empleador
PUT	/trabajos/{id_trabajo}	Administrador, Empleador
DELETE	/trabajos/{id_trabajo}	Administrador, Empleador
Habilidades de trabajos		
GET	/trabajos-habilidades/	Administrador, Moderador, Analista, Empleador, Empleado
GET	/trabajos-habilidades/trabajo/{id_trabajo}	Administrador, Moderador, Analista, Empleador, Empleado
GET	/trabajos-habilidades/{id_trabajo_habilidad}	Administrador, Moderador, Analista, Empleador, Empleado
POST	/trabajos-habilidades/	Administrador, Empleador
PUT	/trabajos-habilidades/{id_trabajo_habilidad}	Administrador, Empleador
DELETE	/trabajos-habilidades/{id_trabajo_habilidad}	Administrador, Empleador
Ofertas		
GET	/ofertas/	Administrador, Moderador, Analista, Empleador, Empleado
GET	/ofertas/{id_oferta}	Administrador, Moderador, Analista, Empleador, Empleado
POST	/ofertas/	Administrador, Empleado
PUT	/ofertas/{id_oferta}	Administrador, Empleado
DELETE	/ofertas/{id_oferta}	Administrador, Empleado
POST	/ofertas/{id_oferta}/aceptar	Administrador, Empleador
POST	/ofertas/{id_oferta}/retirar	Administrador, Empleado
Contrataciones		
GET	/contrataciones/	Administrador, Moderador, Analista, Empleador, Empleado

Método	Ruta	Quién la usa
GET	/contrataciones/{id_contratacion}	Administrador, Moderador, Analista, Empleador, Empleado
POST	/contrataciones/	Administrador, Moderador
PUT	/contrataciones/{id_contratacion}	Administrador, Moderador
DELETE	/contrataciones/{id_contratacion}	Administrador, Moderador
GET	/contrataciones/{id_contratacion}/pago	Administrador, Moderador, Analista, Empleador, Empleado
POST	/contrataciones/{id_contratacion}/custodia	El cliente del contrato (el empleador)
POST	/contrataciones/{id_contratacion}/finalizar	Administrador, Empleador, Empleado
POST	/contrataciones/{id_contratacion}/cancelar	Administrador, Empleador, Empleado
Calificaciones		
GET	/calificaciones/	Administrador, Moderador, Analista, Empleador, Empleado
GET	/calificaciones/{id_calificacion}	Administrador, Moderador, Analista, Empleador, Empleado
POST	/calificaciones/	Administrador, Empleador, Empleado
PUT	/calificaciones/{id_calificacion}	Administrador, Empleador, Empleado
DELETE	/calificaciones/{id_calificacion}	Administrador, Empleador, Empleado
Chat		
GET	/chats/	Administrador, Moderador, Empleador, Empleado
GET	/chats/{id_chat}	Administrador, Moderador, Empleador, Empleado
POST	/chats/	Administrador, Empleador, Empleado
DELETE	/chats/{id_chat}	Administrador, Empleador, Empleado
Mensajes		
GET	/mensajes/	Administrador, Moderador
GET	/mensajes/chat/{id_chat}	Administrador, Moderador, Empleador, Empleado
PUT	/mensajes/chat/{id_chat}/leidos	Administrador, Empleador, Empleado
GET	/mensajes/{id_mensaje}	Administrador, Moderador, Empleador, Empleado
POST	/mensajes/	Administrador, Empleador, Empleado
PUT	/mensajes/{id_mensaje}	Administrador, Empleador, Empleado
DELETE	/mensajes/{id_mensaje}	Administrador, Empleador, Empleado
Notificaciones		
GET	/notificaciones/	Administrador, Moderador, Analista, Empleador, Empleado
GET	/notificaciones/{id_notificacion}	Administrador, Moderador, Analista, Empleador, Empleado
POST	/notificaciones/	Administrador, Moderador
PUT	/notificaciones/{id_notificacion}	Administrador, Moderador
PUT	/notificaciones/{id_notificacion}/leer	Administrador, Moderador, Empleador, Empleado
DELETE	/notificaciones/{id_notificacion}	Administrador, Moderador, Empleador, Empleado
Dispositivos		

Método	Ruta	Quién la usa
GET	/dispositivos/	Administrador, Moderador, Analista, Empleador, Empleado
GET	/dispositivos/{id_dispositivo}	Administrador, Moderador, Analista, Empleador, Empleado
POST	/dispositivos/	Administrador, Moderador
PUT	/dispositivos/{id_dispositivo}	Administrador, Moderador, Empleador, Empleado
DELETE	/dispositivos/{id_dispositivo}	Administrador, Moderador, Empleador, Empleado
Adjuntos		
GET	/adjuntos/	Administrador, Moderador, Empleador, Empleado
GET	/adjuntos/{id_adjunto}	Administrador, Moderador, Empleador, Empleado
POST	/adjuntos/	Administrador, Empleador, Empleado
PUT	/adjuntos/{id_adjunto}	Administrador, Empleador, Empleado
DELETE	/adjuntos/{id_adjunto}	Administrador, Moderador, Empleador, Empleado
Reportes		
GET	/reportes/	Administrador, Moderador, Analista, Empleador, Empleado
GET	/reportes/{id_reporte}	Administrador, Moderador, Analista, Empleador, Empleado
POST	/reportes/	Administrador, Empleador, Empleado
PUT	/reportes/{id_reporte}	Administrador, Moderador, Empleador, Empleado
PUT	/reportes/{id_reporte}/resolver	Administrador, Moderador
DELETE	/reportes/{id_reporte}	Administrador, Moderador
Tokens de usuario		
GET	/tokens-usuario/	Administrador
GET	/tokens-usuario/{id_token}	Administrador
POST	/tokens-usuario/	Administrador
PUT	/tokens-usuario/{id_token}	Administrador
DELETE	/tokens-usuario/{id_token}	Administrador
Auditoría		
GET	/auditoria/	Administrador
GET	/auditoria/{id_auditoria}	Administrador

4. Detalle de cada ruta

Para cada ruta: qué hace, quién la puede usar, qué va en la ruta, el body con sus campos, un ejemplo que funciona con los datos de prueba y lo que responde. Los ids de los ejemplos son los de DB.sql; los registros nuevos (ej: el trabajo 27) son los que se crearon al probar en orden.

Estado de la API

GET `/`

Comprueba que la API está encendida. No pide token.

Quién la usa: Cualquiera, sin token

EJEMPLO

```
GET /
```

RESPUESTA

```
{
  "mensaje": "La API de InWork se encuentra en linea y funcional"
}
```

Autenticación

/auth — registro, login, token y datos del usuario que inició sesión.

POST `/auth/registro`

Crea una cuenta nueva desde la pantalla de registro. Solo se puede elegir Empleador (4) o Empleado (5).

Quién la usa: Cualquiera, sin token

BODY (JSON) — modelo UsuarioRegistro

Campo	Tipo	Obligatorio	Valores / reglas
id_ciudad	número entero	No	id de /ciudades
nombre	texto	Sí	
apellido	texto	Sí	
correo	texto	Sí	único, formato de correo
password	texto	Sí	mínimo 8 caracteres
telefono	texto	Sí	único
tipo_identificacion	texto	Sí	cedula, pasaporte, permiso de residencia, cedula de extranjeria
numero_identificacion	texto	Sí	único junto con el tipo
fecha_nacimiento	fecha	Sí	AAAA-MM-DD; debe ser mayor de 18 años
foto_perfil	texto	No	
descripcion	texto	No	
direccion	texto	No	
id_rol	número entero	No	registro: 4 Empleador o 5 Empleado (si no se envía queda 5). Administrador en POST /usuarios: 1 a 5

EJEMPLO

```
POST /auth/registro
{
  "nombre": "Laura",
  "apellido": "Diaz Ruiz",
  "correo": "laura.diaz@gmail.com",
  "password": "Inwork2026*",
  "telefono": "3001112233",
  "tipo_identificacion": "cedula",
  "numero_identificacion": "1140999888",
  "fecha_nacimiento": "1998-05-05",
  "id_ciudad": 1,
  "descripcion": "Electricista con 5 anos de experiencia",
  "direccion": "Calle 72 #45-10",
  "id_rol": 5
}
```

RESPUESTA

```
{
  "mensaje": "Usuario registrado exitosamente",
  "id": 21
}
```

POST `/auth/login`

Inicia sesión. Devuelve el token que se manda en todas las demás rutas, más el nombre y el rol para mostrarlos en el frontend.

Quién la usa: Cualquiera, sin token

BODY (JSON) — modelo Login

Campo	Tipo	Obligatorio	Valores / reglas
`correo`	texto	Sí	único, formato de correo
`password`	texto	Sí	mínimo 8 caracteres

EJEMPLO

```
POST /auth/login
{
  "correo": "valentina.mendoza93@gmail.com",
  "password": "Inwork2026*"
}
```

RESPUESTA

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6...",
  "token_type": "bearer",
  "id_usuario": 4,
  "nombre": "Valentina Mendoza Rios",
  "id_rol": 4,
  "rol": "Empleador"
}
```

POST `/auth/token`

Lo mismo que login pero en formulario. Es la ruta que usa el botón Authorize de Swagger (/docs). En username se escribe el correo.

Quién la usa: Cualquiera, sin token

Body: formulario (form-data), no JSON: `username` (el correo) y `password`.

EJEMPLO

```
POST /auth/token
username=kevin.escorcia.elec@gmail.com
password=Inwork2026*
```

RESPUESTA

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6...",
  "token_type": "bearer",
  "id_usuario": 12,
  "nombre": "Kevin Escorcia Rada",
  "id_rol": 5,
  "rol": "Empleado"
}
```

POST /auth/cambiar-rol

Botón "Cambiarme a empleador / empleado". Cambia el rol en la base de datos y devuelve un token NUEVO: el frontend debe guardarlo y volver a cargar el menú. El token anterior deja de servir.

Quién la usa: Empleador, Empleado · **Regla:** Solo entre 4 y 5.

BODY (JSON) — modelo CambiarRol

Campo	Tipo	Obligatorio	Valores / reglas
id_rol	número entero	Sí	1 Administrador, 2 Moderador, 3 Analista, 4 Empleador, 5 Empleado

EJEMPLO (con el token de Laura, la cuenta creada en el registro)

```
POST /auth/cambiar-rol
{
  "id_rol": 4
}
```

RESPUESTA

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6...",
  "token_type": "bearer",
  "id_usuario": 21,
  "nombre": "Laura Diaz Ruiz",
  "id_rol": 4,
  "rol": "Empleador"
}
```

GET /auth/me

Datos de la cuenta que inició sesión (sin contraseña).

Quién la usa: Todos los roles (con token)

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /auth/me
```

RESPUESTA

Un objeto con: `id_usuario, id_rol, rol, id_ciudad, nombre, apellido, foto_perfil, correo, correo_verificado, telefono, tipo_identificacion, numero_identificacion, fecha_nacimiento, descripcion, direccion, promedio_calificacion, total_calificaciones, created, updated, estado`.

GET /auth/mis-modulos

Menú del usuario: los módulos de su rol según la tabla rol_modulo, con la ruta del frontend y los permisos (puede_crear, puede_leer, puede_editar, puede_eliminar). Con esto se arma el menú y se muestran u ocultan botones.

Quién la usa: Todos los roles (con token)

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /auth/mis-modulos
```

RESPUESTA

Una lista. Cada elemento trae: `nombre, descripcion, ruta, puede_crear, puede_leer, puede_editar, puede_eliminar`.

PUT `/auth/cambiar-password`

El usuario cambia su propia contraseña.

Quién la usa: Todos los roles (con token)

BODY (JSON) — modelo CambiarPassword

Campo	Tipo	Obligatorio	Valores / reglas
password_actual	texto	Sí	
password_nueva	texto	Sí	

EJEMPLO (con el token de Laura, la cuenta creada en el registro)

```
PUT /auth/cambiar-password
{
  "password_actual": "Inwork2026*",
  "password_nueva": "NuevaClave2026*"
}
```

RESPUESTA

```
{
  "mensaje": "Contrasena actualizada correctamente"
}
```

Roles

/roles — solo el Administrador.

GET `/roles/`

Lista de roles.

Quién la usa: Administrador

EJEMPLO (con el token de administrador: sebastian.rojas@inwork.com.co)

```
GET /roles/
```

RESPUESTA

Una lista. Cada elemento trae: `id_rol, nombre, descripcion, created, updated, estado`.

GET `/roles/{id_rol}`

Un rol.

Quién la usa: Administrador

En la ruta: `{id_rol}` = id (número)

EJEMPLO (con el token de administrador: sebastian.rojas@inwork.com.co)

```
GET /roles/5
```

RESPUESTA

Un objeto con: `id_rol, nombre, descripcion, created, updated, estado`.

POST /roles/

Crea un rol.

Quién la usa: Administrador

BODY (JSON) — modelo Rol

Campo	Tipo	Obligatorio	Valores / reglas
nombre	texto	Sí	
descripcion	texto	No	

No se envía id_rol: lo pone la base de datos.

EJEMPLO (con el token de administrador: sebastian.rojas@inwork.com.co)

```
POST /roles/
{
  "nombre": "Soporte",
  "descripcion": "Atiende dudas de los usuarios"
}
```

RESPUESTA

```
{
  "mensaje": "Rol registrado exitosamente",
  "id": 6
}
```

PUT /roles/{id_rol}

Edita un rol.

Quién la usa: Administrador

En la ruta: {id_rol} = id (número)

BODY (JSON): los mismos campos que POST /roles/.

EJEMPLO (con el token de administrador: sebastian.rojas@inwork.com.co)

```
PUT /roles/6
{
  "nombre": "Soporte",
  "descripcion": "Atiende dudas y quejas de los usuarios"
}
```

RESPUESTA

```
{
  "mensaje": "Rol actualizado correctamente"
}
```

DELETE /roles/{id_rol}

Desactiva un rol (borrado lógico: estado = false).

Quién la usa: Administrador

En la ruta: {id_rol} = id (número)

EJEMPLO (con el token de administrador: sebastian.rojas@inwork.com.co)

```
DELETE /roles/6
```

RESPUESTA

```
{
  "mensaje": "Rol eliminado correctamente"
}
```

Módulos

/modulos — las opciones del menú. Solo el Administrador.

GET /modulos/

Lista de módulos.

Quién la usa: Administrador

EJEMPLO (con el token de administrador: sebastian.rojas@inwork.com.co)

```
GET /modulos/
```

RESPUESTA

Una lista. Cada elemento trae: `id_modulo, nombre, descripcion, ruta, created, updated, estado`.

GET /modulos/{id_modulo}

Un módulo.

Quién la usa: Administrador

En la ruta: `{id_modulo}` = id (número)

EJEMPLO (con el token de administrador: sebastian.rojas@inwork.com.co)

```
GET /modulos/6
```

RESPUESTA

Un objeto con: `id_modulo, nombre, descripcion, ruta, created, updated, estado`.

POST /modulos/

Crea un módulo.

Quién la usa: Administrador

BODY (JSON) — modelo Modulo

Campo	Tipo	Obligatorio	Valores / reglas
`nombre`	texto	Sí	nombre corto, ej: pagos
`descripcion`	texto	No	
`ruta`	texto	No	ruta del menú en el frontend, ej: /pagos

No se envía `id_modulo`: lo pone la base de datos.

EJEMPLO (con el token de administrador: sebastian.rojas@inwork.com.co)

```
POST /modulos/
{
  "nombre": "ayuda",
  "descripcion": "Centro de ayuda",
  "ruta": "/ayuda"
}
```

RESPUESTA

```
{
  "mensaje": "Modulo registrado exitosamente",
  "id": 18
}
```

PUT `/modulos/{id_modulo}`

Edita un módulo.

Quién la usa: Administrador

En la ruta: `{id_modulo}` = id (número)

BODY (JSON): los mismos campos que POST /modulos/.

EJEMPLO (con el token de administrador: sebastian.rojas@inwork.com.co)

```
PUT /modulos/18
{
  "nombre": "ayuda",
  "descripcion": "Preguntas frecuentes",
  "ruta": "/ayuda"
}
```

RESPUESTA

```
{
  "mensaje": "Modulo actualizado correctamente"
}
```

DELETE `/modulos/{id_modulo}`

Desactiva un módulo.

Quién la usa: Administrador

En la ruta: `{id_modulo}` = id (número)

EJEMPLO (con el token de administrador: sebastian.rojas@inwork.com.co)

```
DELETE /modulos/18
```

RESPUESTA

```
{
  "mensaje": "Modulo eliminado correctamente"
}
```

Permisos por rol

/rol-modulo — qué puede hacer cada rol en cada módulo. Solo el Administrador.

GET `/rol-modulo/`

Todos los permisos.

Quién la usa: Administrador

EJEMPLO (con el token de administrador: sebastian.rojas@inwork.com.co)

```
GET /rol-modulo/
```

RESPUESTA

Una lista. Cada elemento trae: `id_rol_modulo, id_rol, id_modulo, puede_crear, puede_leer, puede_editar, puede_eliminar, created, updated, estado`.

GET /rol-modulo/{id_rol_modulo}

Un permiso.

Quién la usa: Administrador

En la ruta: `{id_rol_modulo}` = id (número)

EJEMPLO (con el token de administrador: sebastian.rojas@inwork.com.co)

```
GET /rol-modulo/1
```

RESPUESTA

Un objeto con: `id_rol_modulo, id_rol, id_modulo, puede_crear, puede_leer, puede_editar, puede_eliminar, created, updated, estado`.

POST /rol-modulo/

Le da a un rol permisos sobre un módulo (ej: que el Analista vea adjuntos).

Quién la usa: Administrador · **Regla:** Un registro por rol y módulo.

BODY (JSON) — modelo RolModulo

Campo	Tipo	Obligatorio	Valores / reglas
`id_rol`	número entero	Sí	id de /roles
`id_modulo`	número entero	Sí	id de /modulos
`puede_crear`	true / false	Sí	true o false
`puede_leer`	true / false	Sí	true o false
`puede_editar`	true / false	Sí	true o false
`puede_eliminar`	true / false	Sí	true o false

No se envía `id_rol_modulo`: lo pone la base de datos.

EJEMPLO (con el token de administrador: sebastian.rojas@inwork.com.co)

```
POST /rol-modulo/
{
  "id_rol": 3,
  "id_modulo": 12,
  "puede_crear": false,
  "puede_leer": true,
  "puede_editar": false,
  "puede_eliminar": false
}
```

RESPUESTA

```
{
  "mensaje": "Permiso registrado exitosamente",
  "id": 69
}
```

PUT /rol-modulo/{id_rol_modulo}

Cambia los permisos.

Quién la usa: Administrador

En la ruta: `{id_rol_modulo}` = id (número)

BODY (JSON): los mismos campos que POST /rol-modulo/.

EJEMPLO (con el token de administrador: sebastian.rojas@inwork.com.co)

```
PUT /rol-modulo/69
{
  "id_rol": 3,
  "id_modulo": 12,
  "puede_crear": false,
  "puede_leer": true,
  "puede_editar": false,
  "puede_eliminar": false
}
```

RESPUESTA

```
{
  "mensaje": "Permiso actualizado correctamente"
}
```

DELETE `/rol-modulo/{id_rol_modulo}`

Quita el permiso.

Quién la usa: Administrador

En la ruta: `{id_rol_modulo}` = id (número)

EJEMPLO (con el token de administrador: sebastian.rojas@inwork.com.co)

```
DELETE /rol-modulo/69
```

RESPUESTA

```
{
  "mensaje": "Permiso eliminado correctamente"
}
```

Países

/paises — módulo ubicación.

GET `/paises/`

Lista de países.

Quién la usa: Administrador, Moderador, Analista, Empleador, Empleado

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /paises/
```

RESPUESTA

Una lista. Cada elemento trae: `id_pais, nombre, created, updated, estado`.

GET `/paises/{id_pais}`

Un país.

Quién la usa: Administrador, Moderador, Analista, Empleador, Empleado

En la ruta: `{id_pais}` = id (número)

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /paises/1
```

RESPUESTA

Un objeto con: `id_pais, nombre, created, updated, estado`.

POST /paises/

Crea un país.

Quién la usa: Administrador, Moderador

BODY (JSON) — modelo Pais

Campo	Tipo	Obligatorio	Valores / reglas
nombre	texto	Sí	

No se envía id_pais: lo pone la base de datos.

EJEMPLO (con el token de moderador: mariana.castro@inwork.com.co)

```
POST /paises/
{
  "nombre": "Uruguay"
}
```

RESPUESTA

```
{
  "mensaje": "Pais registrado exitosamente",
  "id": 11
}
```

PUT /paises/{id_pais}

Edita un país.

Quién la usa: Administrador, Moderador

En la ruta: {id_pais} = id (número)

BODY (JSON): los mismos campos que POST /paises/.

EJEMPLO (con el token de moderador: mariana.castro@inwork.com.co)

```
PUT /paises/11
{
  "nombre": "Uruguay"
}
```

RESPUESTA

```
{
  "mensaje": "Pais actualizado correctamente"
}
```

DELETE /paises/{id_pais}

Desactiva un país.

Quién la usa: Administrador, Moderador

En la ruta: {id_pais} = id (número)

EJEMPLO (con el token de moderador: mariana.castro@inwork.com.co)

```
DELETE /paises/11
```

RESPUESTA

```
{
  "mensaje": "Pais eliminado correctamente"
}
```

Ciudades

/ciudades — módulo ubicación.

GET /ciudades/

Lista de ciudades.

Quién la usa: Administrador, Moderador, Analista, Empleador, Empleado

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /ciudades/
```

RESPUESTA

Una lista. Cada elemento trae: `id_ciudad, id_pais, nombre, created, updated, estado`.

GET /ciudades/{id_ciudad}

Una ciudad.

Quién la usa: Administrador, Moderador, Analista, Empleador, Empleado

En la ruta: `{id_ciudad}` = id (número)

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /ciudades/1
```

RESPUESTA

Un objeto con: `id_ciudad, id_pais, nombre, created, updated, estado`.

POST /ciudades/

Crea una ciudad.

Quién la usa: Administrador, Moderador · **Regla:** No se repite el nombre en el mismo país.

BODY (JSON) — modelo Ciudad

Campo	Tipo	Obligatorio	Valores / reglas
`id_pais`	número entero	Sí	id de /paises
`nombre`	texto	Sí	

No se envía `id_ciudad`: lo pone la base de datos.

EJEMPLO (con el token de moderador: mariana.castro@inwork.com.co)

```
POST /ciudades/
{
  "id_pais": 1,
  "nombre": "Sincelejo"
}
```

RESPUESTA

```
{
  "mensaje": "Ciudad registrada exitosamente",
  "id": 11
}
```

PUT /ciudades/{id_ciudad}

Edita una ciudad.

Quién la usa: Administrador, Moderador

En la ruta: `{id_ciudad}` = id (número)

BODY (JSON): los mismos campos que POST /ciudades/.

EJEMPLO (con el token de moderador: mariana.castro@inwork.com.co)

```
PUT /ciudades/11
{
  "id_pais": 1,
  "nombre": "Sincelejo"
}
```

RESPUESTA

```
{
  "mensaje": "Ciudad actualizada correctamente"
}
```

DELETE /ciudades/{id_ciudad}

Desactiva una ciudad.

Quién la usa: Administrador, Moderador

En la ruta: `{id_ciudad}` = id (número)

EJEMPLO (con el token de moderador: mariana.castro@inwork.com.co)

```
DELETE /ciudades/11
```

RESPUESTA

```
{
  "mensaje": "Ciudad eliminada correctamente"
}
```

Categorías

/categorias — tipos de trabajo (Plomería, Limpieza...).

GET /categorias/

Lista de categorías.

Quién la usa: Administrador, Moderador, Analista, Empleador, Empleado

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /categorias/
```

RESPUESTA

Una lista. Cada elemento trae: `id_categoria, nombre, descripcion, created, updated, estado`.

GET /categorias/{id_categoria}

Una categoría.

Quién la usa: Administrador, Moderador, Analista, Empleador, Empleado

En la ruta: `{id_categoria}` = id (número)

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /categorias/1
```

RESPUESTA

Un objeto con: `id_categoria, nombre, descripcion, created, updated, estado`.

POST /categorias/

Crea una categoría.

Quién la usa: Administrador, Moderador

BODY (JSON) — modelo Categoria

Campo	Tipo	Obligatorio	Valores / reglas
nombre	texto	Sí	
descripcion	texto	No	

No se envía id_categoria: lo pone la base de datos.

EJEMPLO (con el token de moderador: mariana.castro@inwork.com.co)

```
POST /categorias/
{
  "nombre": "Mascotas",
  "descripcion": "Paseo y cuidado de mascotas"
}
```

RESPUESTA

```
{
  "mensaje": "Categoria registrada exitosamente",
  "id": 11
}
```

PUT /categorias/{id_categoria}

Edita una categoría.

Quién la usa: Administrador, Moderador

En la ruta: {id_categoria} = id (número)

BODY (JSON): los mismos campos que POST /categorias/.

EJEMPLO (con el token de moderador: mariana.castro@inwork.com.co)

```
PUT /categorias/11
{
  "nombre": "Mascotas",
  "descripcion": "Paseo, baño y cuidado de mascotas"
}
```

RESPUESTA

```
{
  "mensaje": "Categoria actualizada correctamente"
}
```

DELETE /categorias/{id_categoria}

Desactiva una categoría.

Quién la usa: Administrador, Moderador

En la ruta: {id_categoria} = id (número)

EJEMPLO (con el token de moderador: mariana.castro@inwork.com.co)

```
DELETE /categorias/11
```

RESPUESTA

```
{
  "mensaje": "Categoria eliminada correctamente"
}
```

Habilidades

/habilidades — oficios que puede tener un trabajador (usa los permisos de categorías).

GET /habilidades/

Lista de habilidades.

Quién la usa: Administrador, Moderador, Analista, Empleador, Empleado

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /habilidades/
```

RESPUESTA

Una lista. Cada elemento trae: `id_habilidad, nombre, descripcion, created, updated, estado`.

GET /habilidades/{id_habilidad}

Una habilidad.

Quién la usa: Administrador, Moderador, Analista, Empleador, Empleado

En la ruta: `{id_habilidad}` = id (número)

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /habilidades/2
```

RESPUESTA

Un objeto con: `id_habilidad, nombre, descripcion, created, updated, estado`.

POST /habilidades/

Crea una habilidad.

Quién la usa: Administrador, Moderador

BODY (JSON) — modelo Habilidad

Campo	Tipo	Obligatorio	Valores / reglas
`nombre`	texto	Sí	
`descripcion`	texto	No	

No se envía `id_habilidad`: lo pone la base de datos.

EJEMPLO (con el token de moderador: mariana.castro@inwork.com.co)

```
POST /habilidades/
{
  "nombre": "Cerrajeria",
  "descripcion": "Apertura y cambio de cerraduras"
}
```

RESPUESTA

```
{
  "mensaje": "Habilidad registrada exitosamente",
  "id": 11
}
```

PUT /habilidades/{id_habilidad}

Edita una habilidad.

Quién la usa: Administrador, Moderador

En la ruta: `{id_habilidad}` = id (número)

BODY (JSON): los mismos campos que POST /habilidades/.

EJEMPLO (con el token de moderador: mariana.castro@inwork.com.co)

```
PUT /habilidades/11
{
  "nombre": "Cerrajeria",
  "descripcion": "Cerraduras y llaves"
}
```

RESPUESTA

```
{
  "mensaje": "Habilidad actualizada correctamente"
}
```

DELETE /habilidades/{id_habilidad}

Desactiva una habilidad.

Quién la usa: Administrador, Moderador

En la ruta: `{id_habilidad}` = id (número)

EJEMPLO (con el token de moderador: mariana.castro@inwork.com.co)

```
DELETE /habilidades/11
```

RESPUESTA

```
{
  "mensaje": "Habilidad eliminada correctamente"
}
```

Usuarios

/usuarios

GET /usuarios/

Lista de todos los usuarios con el nombre de su rol y de su ciudad.

Quién la usa: Administrador, Moderador, Analista

EJEMPLO (con el token de analista: jorge.pertuz@inwork.com.co)

```
GET /usuarios/
```

RESPUESTA

Una lista. Cada elemento trae: `id_usuario, id_rol, rol, id_ciudad, nombre, apellido, foto_perfil, correo, correo_verificado, telefono, tipo_identificacion, numero_identificacion, fecha_nacimiento, descripcion, direccion, promedio_calificacion, total_calificaciones, created, updated, estado, ciudad`.

GET /usuarios/{id_usuario}

Datos completos de un usuario.

Quién la usa: Administrador, Moderador, Analista, Empleador, Empleado · **Regla:** Empleador y empleado solo pueden ver su propia cuenta; los demás roles ven cualquiera.

En la ruta: `{id_usuario}` = id (número)

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /usuarios/12
```

RESPUESTA

Un objeto con: `id_usuario, id_rol, rol, id_ciudad, nombre, apellido, foto_perfil, correo, correo_verificado, telefono, tipo_identificacion, numero_identificacion, fecha_nacimiento, descripcion, direccion, promedio_calificacion, total_calificaciones, created, updated, estado`.

GET /usuarios/{id_usuario}/perfil

Perfil público de cualquier usuario: nombre, foto, descripción, calificación y ciudad. Úsala para mostrar el nombre de otra persona.

Quién la usa: Todos los roles (con token)

En la ruta: `{id_usuario}` = id (número)

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /usuarios/4/perfil
```

RESPUESTA

Un objeto con: `id_usuario, nombre, apellido, foto_perfil, descripcion, promedio_calificacion, total_calificaciones, ciudad`.

POST /usuarios/

El administrador crea un usuario con cualquier rol (ej: un moderador). Mismo body que el registro.

Quién la usa: Administrador

BODY (JSON): los mismos campos que POST /auth/registro.

EJEMPLO (con el token de administrador: sebastian.rojas@inwork.com.co)

```
POST /usuarios/
{
  "nombre": "Andres",
  "apellido": "Mercado Diaz",
  "correo": "andres.mercado@inwork.com.co",
  "password": "Inwork2026*",
  "telefono": "3004445566",
  "tipo_identificacion": "cedula",
  "numero_identificacion": "1143555777",
  "fecha_nacimiento": "1994-08-12",
  "id_ciudad": 1,
  "descripcion": "Moderador de reportes",
  "direccion": "Carrera 46 #80-15",
  "id_rol": 2
}
```

RESPUESTA

```
{
  "mensaje": "Usuario registrado exitosamente",
  "id": 22
}
```

PUT /usuarios/{id_usuario}

Edita los datos personales. El correo, el documento y la contraseña no se cambian aquí.

Quién la usa: Administrador, Empleador, Empleado · **Regla:** Solo tu propia cuenta (Administrador y Moderador pueden editar otras si tienen el permiso).

En la ruta: `{id_usuario}` = id (número)

BODY (JSON) — modelo UsuarioActualizar

Campo	Tipo	Obligatorio	Valores / reglas
`id_ciudad`	número entero	No	id de /ciudades
`nombre`	texto	Sí	
`apellido`	texto	Sí	
`telefono`	texto	Sí	único
`foto_perfil`	texto	No	
`descripcion`	texto	No	
`direccion`	texto	No	

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
PUT /usuarios/12
{
  "id_ciudad": 1,
  "nombre": "Kevin",
  "apellido": "Escorcia Rada",
  "telefono": "3016548923",
  "foto_perfil": null,
  "descripcion": "Tecnico electricista certificado RETIE, 8 anos de experiencia",
  "direccion": "Carrera 8 #45-23, Rebolo"
}
```

RESPUESTA

```
{
  "mensaje": "Usuario actualizado correctamente"
}
```

PUT /usuarios/{id_usuario}/rol

El administrador le cambia el rol a un usuario. El usuario tendrá que iniciar sesión de nuevo.

Quién la usa: Administrador · **Regla:** No puede cambiar su propio rol.

En la ruta: `{id_usuario}` = id (número)

BODY (JSON): los mismos campos que POST /auth/cambiar-rol.

EJEMPLO (con el token de administrador: sebastian.rojas@inwork.com.co)

```
PUT /usuarios/22/rol
{
  "id_rol": 3
}
```

RESPUESTA

```
{
  "mensaje": "Rol actualizado correctamente"
}
```

PUT /usuarios/{id_usuario}/password

El administrador le pone una contraseña nueva a cualquier usuario (sin pedir la anterior).

Quién la usa: Administrador

En la ruta: `{id_usuario}` = id (número)

BODY (JSON) — modelo RestablecerPassword

Campo	Tipo	Obligatorio	Valores / reglas
password_nueva	texto	Sí	

EJEMPLO (con el token de administrador: sebastian.rojas@inwork.com.co)

```
PUT /usuarios/22/password
{
  "password_nueva": "Temporal2026*"
}
```

RESPUESTA

```
{
  "mensaje": "Contrasena restablecida correctamente"
}
```

DELETE /usuarios/{id_usuario}

Desactiva la cuenta (ya no puede iniciar sesión).

Quién la usa: Administrador, Empleador, Empleado · **Regla:** Solo tu propia cuenta, salvo el Administrador.

En la ruta: `{id_usuario}` = id (número)

EJEMPLO (con el token de administrador: sebastian.rojas@inwork.com.co)

```
DELETE /usuarios/22
```

RESPUESTA

```
{
  "mensaje": "Usuario desactivado correctamente"
}
```

Habilidades de usuarios

/usuarios-habilidades — los oficios de cada trabajador.

GET /usuarios-habilidades/

Todas las habilidades de todos los usuarios.

Quién la usa: Administrador, Moderador, Analista, Empleador, Empleado

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /usuarios-habilidades/
```

RESPUESTA

Una lista. Cada elemento trae: `id_usuario_habilidad, id_usuario, id_habilidad, anios_experiencia, created, updated, estado`.

GET /usuarios-habilidades/{id_usuario_habilidad}

Un registro.

Quién la usa: Administrador, Moderador, Analista, Empleador, Empleado

En la ruta: `{id_usuario_habilidad}` = id (número)

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /usuarios-habilidades/1
```

RESPUESTA

Un objeto con: `id_usuario_habilidad, id_usuario, id_habilidad, anios_experiencia, created, updated, estado`.

POST /usuarios-habilidades/

Agrega una habilidad a TU perfil.

Quién la usa: Administrador, Empleado · **Regla:** No se repite la misma habilidad.

BODY (JSON) — modelo UsuarioHabilidad

Campo	Tipo	Obligatorio	Valores / reglas
`id_habilidad`	número entero	Sí	id de /habilidades
`anios_experiencia`	número entero	Sí	0 o más

No se envía `id_usuario_habilidad`: lo pone la base de datos. `id_usuario` sale del token, no se envía.

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
POST /usuarios-habilidades/
{
  "id_habilidad": 3,
  "anios_experiencia": 2
}
```

RESPUESTA

```
{
  "mensaje": "Habilidad del usuario registrada exitosamente",
  "id": 17
}
```

PUT /usuarios-habilidades/{id_usuario_habilidad}

Edita una habilidad tuya.

Quién la usa: Administrador, Empleador, Empleado · **Regla:** Solo las tuyas.

En la ruta: `{id_usuario_habilidad}` = id (número)

BODY (JSON): los mismos campos que POST /usuarios-habilidades/.

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
PUT /usuarios-habilidades/17
{
  "id_habilidad": 3,
  "anios_experiencia": 3
}
```

RESPUESTA

```
{
  "mensaje": "Habilidad del usuario actualizada correctamente"
}
```

DELETE /usuarios-habilidades/{id_usuario_habilidad}

Quita una habilidad tuya.

Quién la usa: Administrador, Empleador, Empleado · **Regla:** Solo las tuyas.

En la ruta: `{id_usuario_habilidad}` = id (número)

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
DELETE /usuarios-habilidades/17
```

RESPUESTA

```
{
  "mensaje": "Habilidad del usuario eliminada correctamente"
}
```

Verificación de identidad

/verificaciones-identidad — el usuario sube fotos de su documento y un moderador las revisa.

GET /verificaciones-identidad/

Lista de verificaciones.

Quién la usa: Administrador, Moderador, Analista, Empleador, Empleado · **Regla:** Empleador y empleado ven solo las suyas.

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /verificaciones-identidad/
```

RESPUESTA

Una lista. Cada elemento trae: `id_verificacion, id_usuario, url_documento_frente, url_documento_reverso, url_selfie, estado_verificacion, id_admin_revisor, observacion, fecha_revision, created, updated, estado`.

GET /verificaciones-identidad/{id_verificacion}

Una verificación.

Quién la usa: Administrador, Moderador, Analista, Empleador, Empleado · **Regla:** Empleador y empleado solo las suyas.

En la ruta: `{id_verificacion}` = id (número)

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /verificaciones-identidad/9
```

RESPUESTA

Un objeto con: `id_verificacion, id_usuario, url_documento_frente, url_documento_reverso, url_selfie, estado_verificacion, id_admin_revisor, observacion, fecha_revision, created, updated, estado`.

POST /verificaciones-identidad/

Envía los documentos. Queda en estado pendiente.

Quién la usa: Administrador, Empleador, Empleado

BODY (JSON) — modelo VerificacionIdentidad

Campo	Tipo	Obligatorio	Valores / reglas
`url_documento_frente`	texto	Sí	URL de la foto (ej: Cloudinary)
`url_documento_reverso`	texto	No	URL de la foto
`url_selfie`	texto	Sí	URL de la foto

No se envía `id_verificacion`: lo pone la base de datos. `id_usuario` sale del token, no se envía.

EJEMPLO (con el token de Laura, la cuenta creada en el registro)

```
POST /verificaciones-identidad/
{
  "url_documento_frente": "https://res.cloudinary.com/inwork/laura_frente.jpg",
  "url_documento_reverso": "https://res.cloudinary.com/inwork/laura_reverso.jpg",
  "url_selfie": "https://res.cloudinary.com/inwork/laura_selfie.jpg"
}
```

RESPUESTA

```
{
  "mensaje": "Documentos enviados. Un moderador los revisara.",
  "id": 19
}
```

PUT `/verificaciones-identidad/{id_verificacion}`

Cambia las fotos mientras sigue pendiente.

Quién la usa: Administrador, Moderador, Empleador, Empleado · **Regla:** Solo la tuya y si está pendiente.
En la ruta: `{id_verificacion}` = id (número)

BODY (JSON): los mismos campos que POST /verificaciones-identidad/.

EJEMPLO (con el token de Laura, la cuenta creada en el registro)

```
PUT /verificaciones-identidad/19
{
  "url_documento_frente": "https://res.cloudinary.com/inwork/laura_frente2.jpg",
  "url_documento_reverso": "https://res.cloudinary.com/inwork/laura_reverso2.jpg",
  "url_selfie": "https://res.cloudinary.com/inwork/laura_selfie2.jpg"
}
```

RESPUESTA

```
{
  "mensaje": "Verificacion actualizada correctamente"
}
```

PUT `/verificaciones-identidad/{id_verificacion}/revisar`

El moderador aprueba o rechaza.

Quién la usa: Administrador, Moderador
En la ruta: `{id_verificacion}` = id (número)

BODY (JSON) — modelo VerificacionRevisar

Campo	Tipo	Obligatorio	Valores / reglas
`estado_verificacion`	texto	Sí	aprobada, rechazada o pendiente
`observacion`	texto	No	

EJEMPLO (con el token de moderador: mariana.castro@inwork.com.co)

```
PUT /verificaciones-identidad/19/revisar
{
  "estado_verificacion": "aprobada",
  "observacion": "Documentos legibles y coinciden"
}
```

RESPUESTA

```
{
  "mensaje": "Verificacion aprobada"
}
```

DELETE /verificaciones-identidad/{id_verificacion}

Desactiva una verificación.

Quién la usa: Administrador, Moderador

En la ruta: `{id_verificacion}` = id (número)

EJEMPLO (con el token de moderador: mariana.castro@inwork.com.co)

```
DELETE /verificaciones-identidad/19
```

RESPUESTA

```
{
  "mensaje": "Verificacion eliminada correctamente"
}
```

Trabajos

/trabajos — lo que publica el empleador.

GET /trabajos/

Todos los trabajos (los más nuevos primero). Trae estado_trabajo: publicado, asignado, finalizado o cancelado.

Quién la usa: Administrador, Moderador, Analista, Empleador, Empleado

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /trabajos/
```

RESPUESTA

Una lista. Cada elemento trae: `id_trabajo, id_cliente, id_categoria, id_ciudad, titulo, descripcion, direccion, latitud, longitud, fecha_trabajo, hora_inicio, hora_fin, presupuesto, moneda, estado_trabajo, created, updated, estado`.

GET /trabajos/cercanos

Trabajos publicados cerca de un punto, ordenados por distancia. Trae distancia_km.

Quién la usa: Administrador, Moderador, Analista, Empleador, Empleado

Parámetros en la URL (query): `latitud` (obligatorio), `longitud` (obligatorio), `radio_km` (opcional, 10 por defecto)

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /trabajos/cercanos?latitud=10.99&longitud=-74.81&radio_km=5
```

RESPUESTA

Una lista. Cada elemento trae: `id_trabajo, id_cliente, id_categoria, id_ciudad, titulo, descripcion, direccion, latitud, longitud, fecha_trabajo, hora_inicio, hora_fin, presupuesto, moneda, estado_trabajo, created, updated, estado, distancia_km`.

GET /trabajos/{id_trabajo}

Un trabajo.

Quién la usa: Administrador, Moderador, Analista, Empleador, Empleado

En la ruta: `{id_trabajo}` = id (número)

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /trabajos/19
```

RESPUESTA

Un objeto con: `id_trabajo, id_cliente, id_categoria, id_ciudad, titulo, descripcion, direccion, latitud, longitud, fecha_trabajo, hora_inicio, hora_fin, presupuesto, moneda, estado_trabajo, created, updated, estado`.

POST /trabajos/

Publica un trabajo. Queda en estado publicado.

Quién la usa: Administrador, Empleador

BODY (JSON) — modelo Trabajo

Campo	Tipo	Obligatorio	Valores / reglas
id_categoria	número entero	Sí	id de /categorias
id_ciudad	número entero	Sí	id de /ciudades
titulo	texto	Sí	texto corto
descripcion	texto	Sí	
direccion	texto	No	
latitud	número	Sí	entre -90 y 90
longitud	número	Sí	entre -180 y 180
fecha_trabajo	fecha	Sí	AAAA-MM-DD; hoy o una fecha futura
hora_inicio	hora	Sí	HH:MM
hora_fin	hora	Sí	HH:MM; mayor que hora_inicio
presupuesto	número	Sí	mayor que 0
moneda	texto	No	si no se envía queda COP

No se envía `id_trabajo`: lo pone la base de datos. `id_cliente` (el dueño del trabajo) sale del token, no se envía.

EJEMPLO (con el token de empleador: valentina.mendoza93@gmail.com)

```
POST /trabajos/
{
  "id_categoria": 1,
  "id_ciudad": 1,
  "titulo": "Instalar lampara en la sala",
  "descripcion": "Instalar una lampara colgante y revisar el interruptor",
  "direccion": "Calle 98 #52-115, Riomar",
  "latitud": 11.0193,
  "longitud": -74.8337,
  "fecha_trabajo": "2026-11-20",
  "hora_inicio": "08:00",
  "hora_fin": "10:00",
  "presupuesto": 90000,
  "moneda": "COP"
}
```

RESPUESTA

```
{
  "mensaje": "Trabajo publicado exitosamente",
  "id": 27
}
```

PUT /trabajos/{id_trabajo}

Edita un trabajo.

Quién la usa: Administrador, Empleador · **Regla:** Solo el dueño y solo si está publicado.

En la ruta: `{id_trabajo}` = id (número)

BODY (JSON): los mismos campos que POST /trabajos/.

EJEMPLO (con el token de empleador: valentina.mendoza93@gmail.com)

```
PUT /trabajos/27
{
  "id_categoria": 1,
  "id_ciudad": 1,
  "titulo": "Instalar lampara en la sala",
  "descripcion": "Instalar una lampara colgante y revisar el interruptor",
  "direccion": "Calle 98 #52-115, Riomar",
  "latitud": 11.0193,
  "longitud": -74.8337,
  "fecha_trabajo": "2026-11-20",
  "hora_inicio": "08:00",
  "hora_fin": "10:00",
  "presupuesto": 100000,
  "moneda": "COP"
}
```

RESPUESTA

```
{
  "mensaje": "Trabajo actualizado correctamente"
}
```

DELETE `/trabajos/{id_trabajo}`

Desactiva un trabajo.

Quién la usa: Administrador, Empleador · **Regla:** Solo el dueño; no si está asignado.
En la ruta: `{id_trabajo}` = id (número)

EJEMPLO (con el token de empleador: valentina.mendoza93@gmail.com)

```
DELETE /trabajos/28
```

RESPUESTA

```
{
  "mensaje": "Trabajo eliminado correctamente"
}
```

Habilidades de trabajos

/trabajos-habilidades — qué oficio pide cada trabajo.

GET `/trabajos-habilidades/`

Todos los registros.

Quién la usa: Administrador, Moderador, Analista, Empleador, Empleado

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /trabajos-habilidades/
```

RESPUESTA

Una lista. Cada elemento trae: `id_trabajo_habilidad, id_trabajo, id_habilidad, created, updated, estado`.

GET /trabajos-habilidades/trabajo/{id_trabajo}

Las habilidades que pide un trabajo.

Quién la usa: Administrador, Moderador, Analista, Empleador, Empleado

En la ruta: `{id_trabajo}` = id (número)

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /trabajos-habilidades/trabajo/19
```

RESPUESTA

Una lista. Cada elemento trae: `id_trabajo_habilidad, id_trabajo, id_habilidad, created, updated, estado, habilidad`.

GET /trabajos-habilidades/{id_trabajo_habilidad}

Un registro.

Quién la usa: Administrador, Moderador, Analista, Empleador, Empleado

En la ruta: `{id_trabajo_habilidad}` = id (número)

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /trabajos-habilidades/20
```

RESPUESTA

Un objeto con: `id_trabajo_habilidad, id_trabajo, id_habilidad, created, updated, estado`.

POST /trabajos-habilidades/

Le agrega una habilidad a un trabajo tuyo.

Quién la usa: Administrador, Empleador · **Regla:** Solo en tus trabajos.

BODY (JSON) — modelo TrabajoHabilidad

Campo	Tipo	Obligatorio	Valores / reglas
`id_trabajo`	número entero	Sí	un trabajo tuyo
`id_habilidad`	número entero	Sí	id de /habilidades

No se envía `id_trabajo_habilidad`: lo pone la base de datos.

EJEMPLO (con el token de empleador: valentina.mendoza93@gmail.com)

```
POST /trabajos-habilidades/
{
  "id_trabajo": 27,
  "id_habilidad": 2
}
```

RESPUESTA

```
{
  "mensaje": "Habilidad agregada al trabajo",
  "id": 28
}
```

PUT /trabajos-habilidades/{id_trabajo_habilidad}

Cambia la habilidad.

Quién la usa: Administrador, Empleador · **Regla:** Solo en tus trabajos.

En la ruta: `{id_trabajo_habilidad}` = id (número)

BODY (JSON): los mismos campos que POST /trabajos-habilidades/.

EJEMPLO (con el token de empleador: valentina.mendoza93@gmail.com)

```
PUT /trabajos-habilidades/28
{
  "id_trabajo": 27,
  "id_habilidad": 1
}
```

RESPUESTA

```
{
  "mensaje": "Habilidad del trabajo actualizada correctamente"
}
```

DELETE `/trabajos-habilidades/{id_trabajo_habilidad}`

Quita la habilidad.

Quién la usa: Administrador, Empleador · **Regla:** Solo en tus trabajos.

En la ruta: `{id_trabajo_habilidad}` = id (número)

EJEMPLO (con el token de empleador: valentina.mendoza93@gmail.com)

```
DELETE /trabajos-habilidades/28
```

RESPUESTA

```
{
  "mensaje": "Habilidad quitada del trabajo"
}
```

Ofertas

/ofertas — el empleado ofrece hacer un trabajo por un valor.

GET `/ofertas/`

Lista de ofertas.

Quién la usa: Administrador, Moderador, Analista, Empleador, Empleado · **Regla:** Empleador y empleado ven solo las suyas (las que hicieron o las de sus trabajos).

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /ofertas/
```

RESPUESTA

Una lista. Cada elemento trae: `id_oferta, id_trabajo, id_trabajador, valor_oferta, moneda, mensaje, fecha_oferta, estado_oferta, created, updated, estado`.

GET `/ofertas/{id_oferta}`

Una oferta.

Quién la usa: Administrador, Moderador, Analista, Empleador, Empleado · **Regla:** La ve quien ofertó, el dueño del trabajo, el Administrador, el Moderador y el Analista.

En la ruta: `{id_oferta}` = id (número)

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /ofertas/2
```

RESPUESTA

Un objeto con: `id_oferta, id_trabajo, id_trabajador, valor_oferta, moneda, mensaje, fecha_oferta, estado_oferta, created, updated, estado, id_cliente, estado_trabajo, titulo`.

POST /ofertas/

Envía una oferta a un trabajo. Queda pendiente.

Quién la usa: Administrador, Empleado

BODY (JSON) — modelo Oferta

Campo	Tipo	Obligatorio	Valores / reglas
id_trabajo	número entero	Sí	trabajo publicado, que no sea tuyo; una oferta por trabajo
valor_oferta	número	Sí	mayor que 0
moneda	texto	No	si no se envía queda COP
mensaje	texto	No	

No se envía `id_oferta`: lo pone la base de datos. `id_trabajador` sale del token, no se envía.

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
POST /ofertas/
{
  "id_trabajo": 27,
  "valor_oferta": 85000,
  "moneda": "COP",
  "mensaje": "Tengo experiencia en instalaciones, puedo ir ese dia"
}
```

RESPUESTA

```
{
  "mensaje": "Oferta enviada exitosamente",
  "id": 38
}
```

PUT /ofertas/{id_oferta}

Cambia el valor o el mensaje.

Quién la usa: Administrador, Empleado · **Regla:** Solo quien ofertó y si está pendiente.

En la ruta: `{id_oferta}` = id (número)

BODY (JSON): los mismos campos que POST /ofertas/.

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
PUT /ofertas/38
{
  "id_trabajo": 27,
  "valor_oferta": 80000,
  "moneda": "COP",
  "mensaje": "Le dejo un mejor precio"
}
```

RESPUESTA

```
{
  "mensaje": "Oferta actualizada correctamente"
}
```

DELETE /ofertas/{id_oferta}

Desactiva una oferta.

Quién la usa: Administrador, Empleado · **Regla:** Solo quien ofertó; no si fue aceptada.

En la ruta: `{id_oferta}` = id (número)

EJEMPLO (con el token de empleado: wilmer.navarro.pintor@gmail.com)

```
DELETE /ofertas/35
```

RESPUESTA

```
{
  "mensaje": "Oferta eliminada correctamente"
}
```

POST /ofertas/{id_oferta}/aceptar

El dueño del trabajo acepta una oferta. En un solo paso: la oferta queda aceptada, las demás rechazadas, el trabajo pasa a asignado, se crea la contratación, se le avisa al trabajador y se crea la custodia del pago en la API de pagos (Express).

Quién la usa: Administrador, Empleador · **Regla:** Solo el dueño del trabajo; la oferta debe estar pendiente.
En la ruta: `{id_oferta}` = id (número)

EJEMPLO (con el token de empleador: valentina.mendoza93@gmail.com)

```
POST /ofertas/38/aceptar
```

RESPUESTA (con la API de pagos encendida)

```
{
  "mensaje": "Oferta aceptada. Se creo el contrato y la custodia del pago. Ya puedes pagar.",
  "id_contratacion": 21,
  "custodia": {
    "mensaje": "Custodia creada. El cliente debe realizar el pago.",
    "id_custodia": 21,
    "codigo": "CUS-1793500000000-512",
    "monto": 80000,
    "estado_custodia": "pendiente_pago"
  }
}
```

Si la API de pagos está apagada, igual crea el contrato y responde:

```
{
  "mensaje": "Oferta aceptada y contrato creado, pero NO se pudo crear la custodia del pago.",
  "id_contratacion": 21,
  "error_api_pagos": {
    "mensaje": "No se pudo conectar con la API de pagos"
  },
  "solucion": "Intenta de nuevo con POST /contrataciones/21/custodia"
}
```

POST /ofertas/{id_oferta}/retirar

Quien ofertó retira su oferta.

Quién la usa: Administrador, Empleado · **Regla:** Solo quien ofertó y si está pendiente.
En la ruta: `{id_oferta}` = id (número)

EJEMPLO (con el token de empleado: wilmer.navarro.pintor@gmail.com)

```
POST /ofertas/29/retirar
```

RESPUESTA

```
{
  "mensaje": "Oferta retirada"
}
```

Contrataciones

/contrataciones — se crean al aceptar una oferta. Las rutas de pago hablan con la API de pagos (Express).

GET /contrataciones/

Lista de contrataciones. Trae estado_contratacion: activa, finalizada o cancelada.

Quién la usa: Administrador, Moderador, Analista, Empleador, Empleado · **Regla:** Empleador y empleado ven solo en las que participan.

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /contrataciones/
```

RESPUESTA

Una lista. Cada elemento trae: `id_contratacion, id_trabajo, id_cliente, id_trabajador, id_oferta, fecha_contratacion, estado_contratacion, created, updated, estado`.

GET /contrataciones/{id_contratacion}

Una contratación.

Quién la usa: Administrador, Moderador, Analista, Empleador, Empleado · **Regla:** Solo quienes participan, Administrador, Moderador y Analista.

En la ruta: `{id_contratacion}` = id (número)

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /contrataciones/11
```

RESPUESTA

Un objeto con: `id_contratacion, id_trabajo, id_cliente, id_trabajador, id_oferta, fecha_contratacion, estado_contratacion, created, updated, estado`.

POST /contrataciones/

Crear una contratación a mano (lo normal es aceptar una oferta).

Quién la usa: Administrador, Moderador

BODY (JSON) — modelo Contratacion

Campo	Tipo	Obligatorio	Valores / reglas
`id_trabajo`	número entero	Sí	
`id_cliente`	número entero	Sí	el dueño del trabajo
`id_trabajador`	número entero	Sí	quien hizo la oferta
`id_oferta`	número entero	Sí	la oferta debe ser de ese trabajo y de ese trabajador

No se envía `id_contratacion`: lo pone la base de datos.

EJEMPLO (con el token de administrador: sebastian.rojas@inwork.com.co)

```
POST /contrataciones/
{
  "id_trabajo": 19,
  "id_cliente": 5,
  "id_trabajador": 14,
  "id_oferta": 30
}
```

RESPUESTA

```
{
  "mensaje": "Contratacion creada",
  "id": 22,
  "custodia_creada": false,
  "api_pagos": {
    "mensaje": "No se pudo conectar con la API de pagos"
  }
}
```

Nota: Con la API de pagos encendida, custodia_creada sale true y api_pagos trae la custodia creada.

PUT /contrataciones/{id_contratacion}

Cambiar el estado a mano.

Quién la usa: Administrador, Moderador

En la ruta: `{id_contratacion}` = id (número)

BODY (JSON) — modelo ContratacionEstado

Campo	Tipo	Obligatorio	Valores / reglas
estado_contratacion	texto	Sí	activa, finalizada o cancelada

EJEMPLO (con el token de administrador: sebastian.rojas@inwork.com.co)

```
PUT /contrataciones/22
{
  "estado_contratacion": "cancelada"
}
```

RESPUESTA

```
{
  "mensaje": "Contratacion actualizada correctamente"
}
```

DELETE /contrataciones/{id_contratacion}

Desactiva una contratación.

Quién la usa: Administrador, Moderador

En la ruta: `{id_contratacion}` = id (número)

EJEMPLO (con el token de administrador: sebastian.rojas@inwork.com.co)

```
DELETE /contrataciones/22
```

RESPUESTA

```
{
  "mensaje": "Contratacion eliminada correctamente"
}
```

GET /contrataciones/{id_contratacion}/pago

Cómo va el dinero: consulta la custodia en la API de pagos (estado_custodia: pendiente_pago, asegurado, en_disputa, liberado, reembolsado, cancelado).

Quién la usa: Administrador, Moderador, Analista, Empleador, Empleado · **Regla:** Solo quienes participan, Administrador, Moderador y Analista.

En la ruta: `{id_contratacion}` = id (número)

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /contrataciones/11/pago
```

RESPUESTA (necesita la API de pagos encendida)

Trae la custodia de la API de pagos con estos campos: `id_custodia, codigo, id_contratacion, correo_cliente, id_cuenta, monto, porcentaje_comision, estado_custodia, confirmado_cliente, confirmado_trabajador, motivo_disputa, estado, created, updated, correo_trabajador, nombre_trabajador`.

Si la API de pagos está apagada responde 503: `{"detail": "No se pudo conectar con la API de pagos"}`

POST `/contrataciones/{id_contratacion}/custodia`

Vuelve a intentar crear la custodia del pago si falló al aceptar la oferta.

Quién la usa: El cliente del contrato (el empleador) · **Regla:** La contratación debe estar activa.

En la ruta: `{id_contratacion}` = id (número)

EJEMPLO (con el token de empleador: valentina.mendoza93@gmail.com)

```
POST /contrataciones/21/custodia
```

RESPUESTA (necesita la API de pagos encendida)

```
{
  "mensaje": "Custodia creada en la API de pagos. El cliente ya puede pagar.",
  "custodia": {
    "mensaje": "Custodia creada. El cliente debe realizar el pago.",
    "id_custodia": 21,
    "codigo": "CUS-1793500000000-512",
    "monto": 80000,
    "estado_custodia": "pendiente_pago"
  }
}
```

Si la API de pagos está apagada responde 400: `{"detail": "No se pudo conectar con la API de pagos"}`

POST `/contrataciones/{id_contratacion}/finalizar`

El cliente finaliza el contrato. Solo deja si el dinero ya fue liberado en la API de pagos (los dos confirmaron). Después se puede calificar.

Quién la usa: Administrador, Empleador, Empleado · **Regla:** Solo el cliente del contrato.

En la ruta: `{id_contratacion}` = id (número)

EJEMPLO (con el token de empleador: valentina.mendoza93@gmail.com)

```
POST /contrataciones/21/finalizar
```

RESPUESTA (necesita la API de pagos encendida)

```
{
  "mensaje": "Contratacion finalizada. Ya se pueden calificar."
}
```

Si la API de pagos está apagada responde 503: `{"detail": "No se pudo conectar con la API de pagos"}`

POST `/contrataciones/{id_contratacion}/cancelar`

El cliente o el trabajador cancelan. El trabajo vuelve a quedar publicado. No deja si ya hay dinero en custodia.

Quién la usa: Administrador, Empleador, Empleado · **Regla:** Solo quienes participan y si está activa.

En la ruta: `{id_contratacion}` = id (número)

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
POST /contrataciones/21/cancelar
```

RESPUESTA (necesita la API de pagos encendida)

```
{
  "mensaje": "Contratacion cancelada. El trabajo vuelve a estar publicado."
}
```

Si la API de pagos está apagada responde 503: `{"detail": "No se pudo conectar con la API de pagos"}`

Calificaciones

/calificaciones — al terminar, cada uno califica al otro de 1 a 5. El promedio del usuario se actualiza solo.

GET /calificaciones/

Todas las calificaciones.

Quién la usa: Administrador, Moderador, Analista, Empleador, Empleado

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /calificaciones/
```

RESPUESTA

Una lista. Cada elemento trae: `id_calificacion, id_calificador, id_calificado, id_trabajo, puntuacion, comentario, created, updated, estado`.

GET /calificaciones/{id_calificacion}

Una calificación.

Quién la usa: Administrador, Moderador, Analista, Empleador, Empleado

En la ruta: `{id_calificacion}` = id (número)

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /calificaciones/3
```

RESPUESTA

Un objeto con: `id_calificacion, id_calificador, id_calificado, id_trabajo, puntuacion, comentario, created, updated, estado`.

POST /calificaciones/

Califica a la otra persona del contrato.

Quién la usa: Administrador, Empleador, Empleado · **Regla:** La contratación del trabajo debe estar finalizada.

BODY (JSON) — modelo Calificacion

Campo	Tipo	Obligatorio	Valores / reglas
`id_calificado`	número entero	Sí	la otra persona del contrato
`id_trabajo`	número entero	Sí	trabajo con contratación finalizada; una calificación por trabajo
`puntuacion`	número entero	Sí	de 1 a 5
`comentario`	texto	No	

No se envía `id_calificacion`: lo pone la base de datos. `id_calificador` sale del token, no se envía.

EJEMPLO (con el token de empleador: valentina.mendoza93@gmail.com)

```
POST /calificaciones/
{
  "id_calificado": 12,
  "id_trabajo": 27,
  "puntuacion": 5,
  "comentario": "Muy puntual y dejo todo limpio"
}
```

RESPUESTA

```
{
  "mensaje": "Calificacion registrada exitosamente",
  "id": 21
}
```

PUT /calificaciones/{id_calificacion}

Edita tu calificación.

Quién la usa: Administrador, Empleador, Empleado · **Regla:** Solo la tuya.

En la ruta: `{id_calificacion}` = id (número)

BODY (JSON): los mismos campos que POST /calificaciones/.

EJEMPLO (con el token de empleador: valentina.mendoza93@gmail.com)

```
PUT /calificaciones/21
{
  "id_calificado": 12,
  "id_trabajo": 27,
  "puntuacion": 4,
  "comentario": "Buen trabajo, llego un poco tarde"
}
```

RESPUESTA

```
{
  "mensaje": "Calificacion actualizada correctamente"
}
```

DELETE /calificaciones/{id_calificacion}

Desactiva tu calificación.

Quién la usa: Administrador, Empleador, Empleado · **Regla:** Solo la tuya.

En la ruta: `{id_calificacion}` = id (número)

EJEMPLO (con el token de empleador: valentina.mendoza93@gmail.com)

```
DELETE /calificaciones/21
```

RESPUESTA

```
{
  "mensaje": "Calificacion eliminada correctamente"
}
```

Chat

/chats — conversación entre el dueño de un trabajo y un trabajador.

GET /chats/

Lista de chats.

Quién la usa: Administrador, Moderador, Empleador, Empleado · **Regla:** Empleador y empleado ven solo los suyos; Administrador y Moderador ven todos.

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /chats/
```

RESPUESTA

Una lista. Cada elemento trae: `id_chat, id_trabajo, id_cliente, id_trabajador, created, updated, estado`.

GET /chats/{id_chat}

Un chat.

Quién la usa: Administrador, Moderador, Empleador, Empleado · **Regla:** Solo quienes participan o el staff.
En la ruta: `{id_chat}` = id (número)

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /chats/2
```

RESPUESTA

Un objeto con: `id_chat, id_trabajo, id_cliente, id_trabajador, created, updated, estado`.

POST /chats/

Abre un chat.

Quién la usa: Administrador, Empleador, Empleado · **Regla:** Tienes que ser el cliente o el trabajador del chat.

BODY (JSON) — modelo Chat

Campo	Tipo	Obligatorio	Valores / reglas
`id_trabajo`	número entero	Sí	id del trabajo; un chat por trabajo y trabajador
`id_cliente`	número entero	Sí	el dueño del trabajo
`id_trabajador`	número entero	Sí	el trabajador

No se envía `id_chat`: lo pone la base de datos.

EJEMPLO (con el token de empleador: valentina.mendoza93@gmail.com)

```
POST /chats/
{
  "id_trabajo": 23,
  "id_cliente": 4,
  "id_trabajador": 13
}
```

RESPUESTA

```
{
  "mensaje": "Chat creado exitosamente",
  "id": 21
}
```

DELETE /chats/{id_chat}

Desactiva un chat.

Quién la usa: Administrador, Empleador, Empleado · **Regla:** Solo quienes participan.
En la ruta: `{id_chat}` = id (número)

EJEMPLO (con el token de empleador: valentina.mendoza93@gmail.com)

```
DELETE /chats/21
```

RESPUESTA

```
{
  "mensaje": "Chat eliminado correctamente"
}
```

Mensajes

/mensajes — los mensajes de cada chat.

GET /mensajes/

Todos los mensajes de la plataforma.

Quién la usa: Administrador, Moderador

EJEMPLO (con el token de moderador: mariana.castro@inwork.com.co)

```
GET /mensajes/
```

RESPUESTA

Una lista. Cada elemento trae: `id_mensaje, id_chat, id_remitente, mensaje, leido, created, updated, estado`.

GET /mensajes/chat/{id_chat}

Los mensajes de un chat, en orden.

Quién la usa: Administrador, Moderador, Empleador, Empleado · **Regla:** Solo quienes participan o el staff.

En la ruta: `{id_chat}` = id (número)

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /mensajes/chat/2
```

RESPUESTA

Una lista. Cada elemento trae: `id_mensaje, id_chat, id_remitente, mensaje, leido, created, updated, estado`.

PUT /mensajes/chat/{id_chat}/leidos

Marca como leídos los mensajes que te mandaron en ese chat.

Quién la usa: Administrador, Empleador, Empleado · **Regla:** Solo quienes participan.

En la ruta: `{id_chat}` = id (número)

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
PUT /mensajes/chat/2/leidos
```

RESPUESTA

```
{
  "mensaje": "Mensajes marcados como leidos",
  "total": 0
}
```

GET /mensajes/{id_mensaje}

Un mensaje.

Quién la usa: Administrador, Moderador, Empleador, Empleado · **Regla:** Solo quienes participan.

En la ruta: `{id_mensaje}` = id (número)

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /mensajes/4
```

RESPUESTA

Un objeto con: `id_mensaje, id_chat, id_remitente, mensaje, leido, created, updated, estado`.

POST /mensajes/

Envía un mensaje.

Quién la usa: Administrador, Empleador, Empleado · **Regla:** Solo en chats donde participas.

BODY (JSON) — modelo Mensaje

Campo	Tipo	Obligatorio	Valores / reglas
id_chat	número entero	Sí	un chat en el que participas
mensaje	texto	Sí	

No se envía `id_mensaje`: lo pone la base de datos. `id_remitente` sale del token, no se envía.

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
POST /mensajes/
{
  "id_chat": 2,
  "mensaje": "Buenas, puedo ir manana a las 8"
}
```

RESPUESTA

```
{
  "mensaje": "Mensaje enviado",
  "id": 41
}
```

PUT /mensajes/{id_mensaje}

Edita tu mensaje.

Quién la usa: Administrador, Empleador, Empleado · **Regla:** Solo los tuyos.

En la ruta: `{id_mensaje}` = id (número)

BODY (JSON): los mismos campos que POST /mensajes/.

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
PUT /mensajes/41
{
  "id_chat": 2,
  "mensaje": "Buenas, puedo ir manana a las 9"
}
```

RESPUESTA

```
{
  "mensaje": "Mensaje editado correctamente"
}
```

DELETE /mensajes/{id_mensaje}

Desactiva tu mensaje.

Quién la usa: Administrador, Empleador, Empleado · **Regla:** Solo los tuyos.

En la ruta: `{id_mensaje}` = id (número)

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
DELETE /mensajes/41
```

RESPUESTA

```
{
  "mensaje": "Mensaje eliminado correctamente"
}
```

Notificaciones

/notificaciones — avisos para cada usuario (la API también las crea sola, por ejemplo al aceptar una oferta).

GET /notificaciones/

Lista de notificaciones.

Quién la usa: Administrador, Moderador, Analista, Empleador, Empleado · **Regla:** Cada usuario ve las suyas; Administrador y Moderador ven todas.

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /notificaciones/
```

RESPUESTA

Una lista. Cada elemento trae: `id_notificacion, id_usuario, tipo, titulo, contenido, id_referencia, leida, created, updated, estado`.

GET /notificaciones/{id_notificacion}

Una notificación.

Quién la usa: Administrador, Moderador, Analista, Empleador, Empleado · **Regla:** Solo las tuyas (o el staff).
En la ruta: `{id_notificacion}` = id (número)

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /notificaciones/2
```

RESPUESTA

Un objeto con: `id_notificacion, id_usuario, tipo, titulo, contenido, id_referencia, leida, created, updated, estado`.

POST /notificaciones/

Envía una notificación a un usuario.

Quién la usa: Administrador, Moderador

BODY (JSON) — modelo Notificacion

Campo	Tipo	Obligatorio	Valores / reglas
`id_usuario`	número entero	Sí	a quién le llega
`tipo`	texto	Sí	texto: oferta_aceptada, contratacion_finalizada, contratacion_cancelada, verificacion, reporte, sistema
`titulo`	texto	Sí	texto corto
`contenido`	texto	No	
`id_referencia`	número entero	No	id del registro relacionado (opcional)

No se envía `id_notificacion`: lo pone la base de datos.

EJEMPLO (con el token de moderador: mariana.castro@inwork.com.co)

```
POST /notificaciones/
{
  "id_usuario": 12,
  "tipo": "sistema",
  "titulo": "Bienvenido a InWork",
  "contenido": "Completa tu perfil para recibir mas trabajos",
  "id_referencia": null
}
```

RESPUESTA

```
{
  "mensaje": "Notificacion creada exitosamente",
  "id": 38
}
```

PUT /notificaciones/{id_notificacion}

Edita una notificación.

Quién la usa: Administrador, Moderador

En la ruta: {id_notificacion} = id (número)

BODY (JSON): los mismos campos que POST /notificaciones/.

EJEMPLO (con el token de moderador: mariana.castro@inwork.com.co)

```
PUT /notificaciones/38
{
  "id_usuario": 12,
  "tipo": "sistema",
  "titulo": "Bienvenido a InWork",
  "contenido": "Completa tu perfil y verifica tu identidad",
  "id_referencia": null
}
```

RESPUESTA

```
{
  "mensaje": "Notificacion actualizada correctamente"
}
```

PUT /notificaciones/{id_notificacion}/leer

Marca la notificación como leída.

Quién la usa: Administrador, Moderador, Empleador, Empleado · **Regla:** Solo las tuyas.

En la ruta: {id_notificacion} = id (número)

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
PUT /notificaciones/38/leer
```

RESPUESTA

```
{
  "mensaje": "Notificacion marcada como leida"
}
```

DELETE /notificaciones/{id_notificacion}

Desactiva una notificación.

Quién la usa: Administrador, Moderador, Empleador, Empleado · **Regla:** Solo las tuyas.

En la ruta: {id_notificacion} = id (número)

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
DELETE /notificaciones/38
```

RESPUESTA

```
{
  "mensaje": "Notificacion eliminada correctamente"
}
```

Dispositivos

/dispositivos — celulares o navegadores donde le llegan las notificaciones push (usa los permisos de notificaciones).

GET /dispositivos/

Lista de dispositivos.

Quién la usa: Administrador, Moderador, Analista, Empleador, Empleado · **Regla:** Cada usuario ve los suyos; Administrador y Moderador ven todos.

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /dispositivos/
```

RESPUESTA

Una lista. Cada elemento trae: `id_dispositivo, id_usuario, token_push, plataforma, created, updated, estado`.

GET /dispositivos/{id_dispositivo}

Un dispositivo.

Quién la usa: Administrador, Moderador, Analista, Empleador, Empleado · **Regla:** Solo los tuyos (o el staff).
En la ruta: `{id_dispositivo}` = id (número)

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
GET /dispositivos/6
```

RESPUESTA

Un objeto con: `id_dispositivo, id_usuario, token_push, plataforma, created, updated, estado`.

POST /dispositivos/

Registra tu dispositivo.

Quién la usa: Administrador, Moderador

BODY (JSON) — modelo Dispositivo

Campo	Tipo	Obligatorio	Valores / reglas
`token_push`	texto	Sí	único (token de Firebase u otro)
`plataforma`	texto	Sí	android, ios o web

No se envía `id_dispositivo`: lo pone la base de datos. `id_usuario` sale del token, no se envía.

EJEMPLO (con el token de moderador: mariana.castro@inwork.com.co)

```
POST /dispositivos/
{
  "token_push": "fcm_mariana_web_8f3a2c",
  "plataforma": "web"
}
```

RESPUESTA

```
{
  "mensaje": "Dispositivo registrado exitosamente",
  "id": 11
}
```

PUT /dispositivos/{id_dispositivo}

Actualiza el token push.

Quién la usa: Administrador, Moderador, Empleador, Empleado · **Regla:** Solo los tuyos.

En la ruta: `{id_dispositivo}` = id (número)

BODY (JSON): los mismos campos que POST /dispositivos/.

EJEMPLO (con el token de moderador: mariana.castro@inwork.com.co)

```
PUT /dispositivos/11
{
  "token_push": "fcm_mariana_web_9b1d4e",
  "plataforma": "web"
}
```

RESPUESTA

```
{
  "mensaje": "Dispositivo actualizado correctamente"
}
```

DELETE /dispositivos/{id_dispositivo}

Quita un dispositivo.

Quién la usa: Administrador, Moderador, Empleador, Empleado · **Regla:** Solo los tuyos.

En la ruta: `{id_dispositivo}` = id (número)

EJEMPLO (con el token de moderador: mariana.castro@inwork.com.co)

```
DELETE /dispositivos/11
```

RESPUESTA

```
{
  "mensaje": "Dispositivo eliminado correctamente"
}
```

Adjuntos

/adjuntos — fotos o archivos de un trabajo, mensaje o reporte (se guarda la URL).

GET /adjuntos/

Todos los adjuntos.

Quién la usa: Administrador, Moderador, Empleador, Empleado

EJEMPLO (con el token de empleador: valentina.mendoza93@gmail.com)

```
GET /adjuntos/
```

RESPUESTA

Una lista. Cada elemento trae: `id_adjunto, tipo_entidad, id_entidad, id_subido_por, url, tipo_mime, created, updated, estado`.

GET /adjuntos/{id_adjunto}

Un adjunto.

Quién la usa: Administrador, Moderador, Empleador, Empleado

En la ruta: `{id_adjunto}` = id (número)

EJEMPLO (con el token de empleador: valentina.mendoza93@gmail.com)

```
GET /adjuntos/1
```

RESPUESTA

Un objeto con: `id_adjunto, tipo_entidad, id_entidad, id_subido_por, url, tipo_mime, created, updated, estado`.

POST /adjuntos/

Registra un adjunto.

Quién la usa: Administrador, Empleador, Empleado

BODY (JSON) — modelo Adjunto

Campo	Tipo	Obligatorio	Valores / reglas
`tipo_entidad`	texto	Sí	trabajo, mensaje o reporte
`id_entidad`	número entero	Sí	id del trabajo, mensaje o reporte
`url`	texto	Sí	URL del archivo
`tipo_mime`	texto	No	ej: image/jpeg, application/pdf

No se envía `id_adjunto`: lo pone la base de datos. `id_subido_por` sale del token, no se envía.

EJEMPLO (con el token de empleador: valentina.mendoza93@gmail.com)

```
POST /adjuntos/
{
  "tipo_entidad": "trabajo",
  "id_entidad": 27,
  "url": "https://res.cloudinary.com/inwork/lampara.jpg",
  "tipo_mime": "image/jpeg"
}
```

RESPUESTA

```
{
  "mensaje": "Adjunto registrado exitosamente",
  "id": 11
}
```

PUT /adjuntos/{id_adjunto}

Edita un adjunto.

Quién la usa: Administrador, Empleador, Empleado · **Regla:** Solo quien lo subió.

En la ruta: `{id_adjunto}` = id (número)

BODY (JSON): los mismos campos que POST /adjuntos/.

EJEMPLO (con el token de empleador: valentina.mendoza93@gmail.com)

```
PUT /adjuntos/11
{
  "tipo_entidad": "trabajo",
  "id_entidad": 27,
  "url": "https://res.cloudinary.com/inwork/lampara_sala.jpg",
  "tipo_mime": "image/jpeg"
}
```

RESPUESTA

```
{
  "mensaje": "Adjunto actualizado correctamente"
}
```

DELETE /adjuntos/{id_adjunto}

Desactiva un adjunto.

Quién la usa: Administrador, Moderador, Empleador, Empleado · **Regla:** Solo quien lo subió (o el staff).

En la ruta: `{id_adjunto}` = id (número)

EJEMPLO (con el token de empleador: valentina.mendoza93@gmail.com)

```
DELETE /adjuntos/11
```

RESPUESTA

```
{
  "mensaje": "Adjunto eliminado correctamente"
}
```

Reportes

/reportes — un usuario denuncia a otro y el moderador lo revisa.

GET /reportes/

Lista de reportes.

Quién la usa: Administrador, Moderador, Analista, Empleador, Empleado · **Regla:** Empleador y empleado ven solo los que hicieron.

EJEMPLO (con el token de analista: jorge.pertuz@inwork.com.co)

```
GET /reportes/
```

RESPUESTA

Una lista. Cada elemento trae: `id_reporte, id_reportante, id_reportado, id_trabajo, motivo, descripcion, estado_reporte, id_admin_revisor, resolucion, fecha_resolucion, created, updated, estado`.

GET /reportes/{id_reporte}

Un reporte.

Quién la usa: Administrador, Moderador, Analista, Empleador, Empleado · **Regla:** Empleador y empleado solo los suyos.

En la ruta: `{id_reporte}` = id (número)

EJEMPLO (con el token de empleador: valentina.mendoza93@gmail.com)

```
GET /reportes/10
```

RESPUESTA

Un objeto con: `id_reporte, id_reportante, id_reportado, id_trabajo, motivo, descripcion, estado_reporte, id_admin_revisor, resolucion, fecha_resolucion, created, updated, estado`.

POST /reportes/

Reporta a un usuario por algo que pasó en un trabajo. Queda pendiente.

Quién la usa: Administrador, Empleador, Empleado

BODY (JSON) — modelo Reporte

Campo	Tipo	Obligatorio	Valores / reglas
`id_reportado`	número entero	Sí	usuario relacionado con el trabajo
`id_trabajo`	número entero	Sí	trabajo donde los dos participan; un reporte por persona y trabajo
`motivo`	texto	Sí	fraude, incumplimiento, lenguaje_inapropiado, estafa, suplantacion, otro
`descripcion`	texto	Sí	

No se envía `id_reporte`: lo pone la base de datos. `id_reportante` sale del token, no se envía.

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
POST /reportes/
{
  "id_reportado": 4,
  "id_trabajo": 27,
  "motivo": "incumplimiento",
  "descripcion": "Cambio la hora del trabajo sin avisar"
}
```

RESPUESTA

```
{
  "mensaje": "Reporte enviado exitosamente",
  "id": 11
}
```

PUT `/reportes/{id_reporte}`

Edita tu reporte mientras está pendiente.

Quién la usa: Administrador, Moderador, Empleador, Empleado · **Regla:** Solo el tuyo y si está pendiente.

En la ruta: `{id_reporte}` = id (número)

BODY (JSON): los mismos campos que POST /reportes/.

EJEMPLO (con el token de empleado: kevin.escorcia.elec@gmail.com)

```
PUT /reportes/11
{
  "id_reportado": 4,
  "id_trabajo": 27,
  "motivo": "incumplimiento",
  "descripcion": "Cambio la hora del trabajo dos veces sin avisar"
}
```

RESPUESTA

```
{
  "mensaje": "Reporte actualizado correctamente"
}
```

PUT `/reportes/{id_reporte}/resolver`

El moderador cambia el estado y escribe la resolución.

Quién la usa: Administrador, Moderador

En la ruta: `{id_reporte}` = id (número)

BODY (JSON) — modelo ReporteResolver

Campo	Tipo	Obligatorio	Valores / reglas
estado_reporte	texto	Sí	pendiente, en_revision, resuelto o rechazado
resolucion	texto	No	

EJEMPLO (con el token de moderador: mariana.castro@inwork.com.co)

```
PUT /reportes/11/resolver
{
  "estado_reporte": "resuelto",
  "resolucion": "Se hablo con las dos partes y se acordo nueva fecha"
}
```

RESPUESTA

```
{
  "mensaje": "Reporte actualizado"
}
```

DELETE /reportes/{id_reporte}

Desactiva un reporte.

Quién la usa: Administrador, Moderador

En la ruta: `{id_reporte}` = id (número)

EJEMPLO (con el token de moderador: mariana.castro@inwork.com.co)

```
DELETE /reportes/11
```

RESPUESTA

```
{
  "mensaje": "Reporte eliminado correctamente"
}
```

Tokens de usuario

/tokens-usuario — códigos para verificar correo o recuperar contraseña (módulo seguridad, solo el Administrador).

GET /tokens-usuario/

Lista de tokens.

Quién la usa: Administrador

EJEMPLO (con el token de administrador: sebastian.rojas@inwork.com.co)

```
GET /tokens-usuario/
```

RESPUESTA

Una lista. Cada elemento trae: `id_token, id_usuario, tipo, expira, usado, created, updated, estado`.

GET /tokens-usuario/{id_token}

Un token.

Quién la usa: Administrador

En la ruta: `{id_token}` = id (número)

EJEMPLO (con el token de administrador: sebastian.rojas@inwork.com.co)

```
GET /tokens-usuario/1
```

RESPUESTA

Un objeto con: `id_token, id_usuario, tipo, expira, usado, created, updated, estado`.

POST /tokens-usuario/

Crea un token.

Quién la usa: Administrador

BODY (JSON) — modelo TokenUsuario

Campo	Tipo	Obligatorio	Valores / reglas
`id_usuario`	número entero	Sí	
`tipo`	texto	Sí	verificacion_correo, recuperar_password o refresh
`token`	texto	Sí	
`expira`	fecha y hora	Sí	fecha y hora: AAAA-MM-DDTHH:MM:SS
`usado`	true / false	No	true o false (si no se envía queda false)

No se envía `id_token`: lo pone la base de datos.

EJEMPLO (con el token de administrador: sebastian.rojas@inwork.com.co)

```
POST /tokens-usuario/
{
  "id_usuario": 12,
  "tipo": "recuperar_password",
  "token": "a8f3c2e9d1b74f6a",
  "expira": "2026-11-30T23:59:00",
  "usado": false
}
```

RESPUESTA

```
{
  "mensaje": "Token registrado exitosamente",
  "id": 11
}
```

PUT `/tokens-usuario/{id_token}`

Edita un token (ej: marcarlo como usado).

Quién la usa: Administrador

En la ruta: `{id_token}` = id (número)

BODY (JSON): los mismos campos que POST /tokens-usuario/.

EJEMPLO (con el token de administrador: sebastian.rojas@inwork.com.co)

```
PUT /tokens-usuario/11
{
  "id_usuario": 12,
  "tipo": "recuperar_password",
  "token": "a8f3c2e9d1b74f6a",
  "expira": "2026-11-30T23:59:00",
  "usado": true
}
```

RESPUESTA

```
{
  "mensaje": "Token actualizado correctamente"
}
```

DELETE `/tokens-usuario/{id_token}`

Desactiva un token.

Quién la usa: Administrador

En la ruta: `{id_token}` = id (número)

EJEMPLO (con el token de administrador: sebastian.rojas@inwork.com.co)

```
DELETE /tokens-usuario/11
```

RESPUESTA

```
{
  "mensaje": "Token eliminado correctamente"
}
```

Auditoría

/auditoria — quién creó, editó o borró qué (la API la llena sola). Solo el Administrador.

GET /auditoria/

Todo el historial.

Quién la usa: Administrador

EJEMPLO (con el token de administrador: sebastian.rojas@inwork.com.co)

```
GET /auditoria/
```

RESPUESTA

Una lista. Cada elemento trae: `id_auditoria, id_usuario, tabla, id_registro, accion, datos_anteriores, datos_nuevos, ip, created, updated, estado`.

GET /auditoria/{id_auditoria}

Un registro del historial.

Quién la usa: Administrador

En la ruta: `{id_auditoria}` = id (número)

EJEMPLO (con el token de administrador: sebastian.rojas@inwork.com.co)

```
GET /auditoria/1
```

RESPUESTA

Un objeto con: `id_auditoria, id_usuario, tabla, id_registro, accion, datos_anteriores, datos_nuevos, ip, created, updated, estado`.